

Frontend Interview Revision Sheet

One sheet to re-read before every interview. Built for recall, not learning — every line is a trigger for something you already know.

How to use this (fixing the "going blank" problem)

Going blank ≠ not knowing. It's retrieval failure under pressure. Fix it with retrieval practice, not re-reading:

1. **Active recall, not passive reading.** Cover the answer column, read the concept name, say the answer OUT LOUD in full sentences. Speaking is what interviews test — silent recognition feels like knowing but isn't.
 2. **Spaced repetition.** Day 1 → Day 2 → Day 4 → Day 7 → before each interview. Only revise the ones you stumbled on.
 3. **The 30-second rule.** For every concept, be able to give a 30-second answer (definition + why it exists + one example). Rehearse that exact shape.
 4. **Blank-out protocol in the interview:** buy time by restating the question ("So you're asking how React decides what to re-render...") — restating almost always triggers the memory. If not, start from the problem the concept solves, not the definition.
-

1. CSS

Display

Value	Behavior
<code>block</code>	Full width, new line, respects width/height/margin
<code>inline</code>	Flows in text, ignores width/height, only horizontal margin/padding push
<code>inline-block</code>	Flows inline BUT respects width/height

Value	Behavior
<code>none</code>	Removed from layout AND accessibility tree (vs <code>visibility: hidden</code> — hidden but still takes space)
<code>flex / grid</code>	Element becomes a flex/grid container ; children become flex/grid items
<code>contents</code>	Element's box disappears, children act as if direct children of the parent

Flexbox (one axis)

Container properties:

- `flex-direction: row | column | row-reverse | column-reverse` — defines the **main axis**
- `justify-content: alignment along main axis` — `flex-start` | `center` | `space-between` | `space-around` | `space-evenly`
- `align-items: alignment along cross axis` — `stretch` (default) | `center` | `flex-start` | `flex-end` | `baseline`
- `align-content: cross-axis alignment of multiple lines` (only matters with `flex-wrap: wrap`)
- `flex-wrap: nowrap` (default) | `wrap`
- `gap`: spacing between items (works in flex and grid)

Item properties:

- `flex-grow`: how much extra space this item takes (ratio, default 0)
- `flex-shrink`: how much it shrinks when space is tight (default 1)
- `flex-basis`: starting size before grow/shrink (default `auto`)
- `flex: 1` $\equiv$ `flex: 1 1 0` — grow 1, shrink 1, basis 0 (equal columns!)
- `align-self`: override `align-items` for one item
- `order`: visual reorder without changing DOM

Classic Q: center a div → `display: flex; justify-content: center; align-items: center;` (know grid version too: `display: grid; place-items: center;`)

Grid (two axes)

- `grid-template-columns: repeat(3, 1fr) / repeat(auto-fill, minmax(200px, 1fr))` (responsive cards without media queries — say this in interviews)
- `fr` = fraction of free space
- `grid-column: 1 / 3` or `span 2` — item placement
- Flex vs Grid: flex = content-out, one dimension; grid = layout-in, two dimensions

Position + inset

- `static` (default) — top/left/inset do nothing
- `relative` — offset from its own normal spot; **creates positioning context for absolute children**
- `absolute` — removed from flow; positioned relative to **nearest positioned ancestor** (else viewport/initial containing block)
- `fixed` — relative to viewport, ignores scroll (gotcha: `transform` on an ancestor makes it the containing block)
- `sticky` — relative until a scroll threshold, then fixed within its parent
- `inset` = shorthand for `top right bottom left`. `inset: 0` $\equiv$ all four = 0 $\rightarrow$ stretches an absolute element to fill its positioned parent (overlay pattern)

Box model & sizing

- content $\rightarrow$ padding $\rightarrow$ border $\rightarrow$ margin
- `box-sizing: content-box` (default): width = content only. `border-box`: width includes padding + border (why everyone sets `* { box-sizing: border-box }`)
- Margin collapse: vertical margins of adjacent blocks merge (larger wins). Doesn't happen in flex/grid.

Specificity & cascade

- Order: inline style > ID > class/attribute/pseudo-class > element. **!important** overrides all (avoid).
- Count as (IDs, classes, elements): `#nav .item a` = (1,1,1)
- Equal specificity $\rightarrow$ later rule wins

Media queries

`@media (max-width: 768px) { ... }` `/* desktop-first */`

```
@media (min-width: 768px) { ... }    /* mobile-first (preferred) */
```

```
@media (prefers-color-scheme: dark) { ... }
```

- Mobile-first = base styles for mobile, `min-width` queries add complexity upward
- Modern range syntax: `@media (width <= 768px)`
- Container queries (`@container`) — component responds to its container's size, not viewport (good modern-CSS mention)

Transform, translate, transition, animation

- `transform: translate(x, y) | translateX() | scale() | rotate() | skew()` — GPU-accelerated, **doesn't trigger layout/reflow** (why you animate transform/opacity, not top/left/width)
- `translate(-50%, -50%) + position: absolute; top: 50%; left: 50% =` classic centering (percent is of the **element's own size**)
- `transition: property duration timing-function delay` → `transition: transform 0.3s ease-in-out`
- `@keyframes + animation: name 2s infinite` for multi-step animation
- Composite-only properties (`transform`, `opacity`) skip layout & paint → 60fps

Misc CSS quick hits

- Units: `rem` (root font-size, use for spacing/typography), `em` (parent font-size), `%`, `vh/vw`, `px`
- Pseudo-classes (`:hover`, `:focus`, `:nth-child(2n)`, `:not()`) vs pseudo-elements (`::before`, `::after` — need `content: ""`)
- `z-index` only works on positioned elements; stacking context is created by `position + z-index`, `opacity < 1`, `transform`, etc. — a child can never escape its parent's stacking context
- Hide accessibly: `display:none` (gone), `visibility:hidden` (space kept), `opacity:0` (interactive!), `.sr-only` clip pattern (screen-reader only)
- BEM naming: `block__element--modifier`

2. JavaScript

Variables & scope

- `var`: function-scoped, hoisted as `undefined`, re-declarable

- `let/const`: block-scoped, hoisted but in **Temporal Dead Zone** (ReferenceError before declaration)
- `const` = binding can't be reassigned; object contents still mutable

Closures

A function "remembers" the variables of the scope it was created in, even after that scope has finished.

```
function counter() {

  let count = 0;

  return () => ++count; // closes over count

}
```

- Interview classic: `for (var i...) setTimeout(...)` prints the final value N times → fix with `let` (new binding per iteration) or an IIFE
- React link: **stale closure** — an effect/callback captured old state because it closed over a previous render's variables

this

- Regular function: `this` = how it's **called** (object before the dot; undefined in strict mode; window otherwise)
- Arrow function: no own `this` — inherits from where it was **defined** (lexical)
- `call(thisArg, a, b)` / `apply(thisArg, [a, b])` invoke immediately; `bind(thisArg)` returns a new bound function

Prototypes

- Every object has an internal `[[Prototype]]` link; property lookup walks up the chain
- `class` is syntactic sugar over prototypes; `extends` sets up the chain
- `Object.create(proto)` makes an object with a given prototype

Event loop

Order per cycle: **synchronous code** → **ALL microtasks** → **one macrotask** → **repeat** (render between).

- Microtasks: Promise `.then`, `queueMicrotask`, `MutationObserver`
- Macrotasks: `setTimeout`, `setInterval`, I/O, events

```

console.log('1');

setTimeout(() => console.log('4'));

Promise.resolve().then(() => console.log('3'));

console.log('2');

// 1 2 3 4 — promise beats timeout, always

```

Promises

States: pending → fulfilled | rejected (settled = either, and it's final).

Combinator	Resolves when	Rejects when	Returns
<code>Promise.all</code>	ALL fulfill	any one rejects (fail-fast)	array of values, input order
<code>Promise.allSettled</code>	all settle	never rejects	array of `{status, value}`
<code>Promise.race</code>	first to settle (fulfil OR reject)	first rejection if it's first	that one result
<code>Promise.any</code>	first to fulfill	all reject → <code>AggregateError</code>	first fulfilled value

- `all` vs `allSettled` (they love this): `all` for "I need every result or the whole thing fails" (parallel required fetches); `allSettled` for "fire them all, handle successes and failures individually" (batch operations, dashboards)
- `async/await` = sugar over promises. `await` pauses the async function (not the thread). `try/catch` replaces `.catch`. Sequential awaits are serial — use `Promise.all` for parallel.
- Unhandled rejection gotcha: starting promises in a loop with `await` inside = serial; map to promises first, then `await Promise.all(promises)`

Copies & equality

- `==` coerces, `===` doesn't (always use `===`; know `null == undefined` is true)
- Shallow copy: spread `{...obj}`, `Object.assign` — nested objects still shared

- Deep copy: `structuredClone(obj)` (modern), or `JSON.parse(JSON.stringify())` (loses functions, Dates, undefined)

Functions patterns (machine-coding favorites)

// Debounce — run after user STOPS (search input)

```
const debounce = (fn, delay) => {

  let t;

  return (...args) => { clearTimeout(t); t = setTimeout(() => fn(...args), delay); };

};
```

// Throttle — run at most once per interval (scroll)

```
const throttle = (fn, limit) => {

  let ok = true;

  return (...args) => { if (ok) { fn(...args); ok = false; setTimeout(() => ok = true, limit); } };

};
```

- Currying: `f(a)(b)(c)` — return functions until all args collected
- IIFE: `(function(){ ... })()` — isolate scope

Array methods (say the return type!)

- `map` → new array (same length), `filter` → new array (subset), `reduce` → single accumulated value
- `find` → first match or undefined, `findIndex` → index or -1
- `some` → boolean (any pass?), `every` → boolean (all pass?)
- `forEach` → undefined (side effects only, can't break)
- `slice` (copy, non-mutating) vs `splice` (mutates, removes/inserts)
- Mutating: push/pop/shift/unshift/splice/sort/reverse — sort gotcha: default is string sort, pass `(a,b) => a-b`

Browser & misc

- Event delegation: one listener on a parent, use `e.target` — works for dynamic children (bubbling)

- `e.stopPropagation()` vs `e.preventDefault()`
 - `localStorage` (persists) vs `sessionStorage` (tab lifetime) vs cookies (sent with requests, ~4KB)
 - ES6 grab-bag: destructuring, template literals, default/rest/spread params, optional chaining `?.`, nullish coalescing `??` (only null/undefined, unlike `||` which catches 0 and "")
 - Modules: `import/export`, static, tree-shakeable — vs CommonJS `require` (dynamic, Node)
-

3. React

Core mental model

- UI = f(state). You describe the UI for a given state; React figures out DOM updates.
- **Virtual DOM**: lightweight JS object tree describing the UI. On state change React builds a new tree, diffs it with the previous one, applies minimal DOM mutations.

Reconciliation

The diffing algorithm. Heuristics that make it $O(n)$:

1. **Different element type** at same position → tear down subtree, rebuild (`<div>` → `<span>` = full remount, state lost)
2. **Same type** → keep DOM node, update changed props, recurse into children
3. **Keys** tell React which list items are "the same" across renders → moved instead of destroyed/recreated. Index-as-key breaks when the list reorders/inserts (state sticks to position, not item).

Fiber

The reconciler architecture (React 16+). 30-second answer:

- Each element gets a **fiber node** (a JS object holding type, props, state, effects, pointers to parent/child/sibling)
- Rendering became **interruptible**: work is split into units, React can pause, yield to the browser (keep input responsive), resume, or throw work away
- Two phases: **render phase** (build the work-in-progress tree — interruptible, no side effects) and **commit phase** (apply DOM changes — synchronous, all at once)
- This is what enables concurrent features: `useTransition`, `Suspense`, priority-based scheduling

How state works

- `useState` returns state + setter. Setter **schedules a re-render**; it does NOT change the variable in the current running code (state is a snapshot per render).
- **Batching**: multiple `setState` calls in one event handler → one re-render. React 18: batching everywhere (timeouts, promises too).
- Updater form for dependent updates: `setCount(c => c + 1)` — three plain `setCount(count + 1)` in a row = +1, three updater-form = +3.
- State updates must be **immutable**: new object/array reference, or React (and memo/deps comparisons) won't see a change.
- Props down, events up. Lift state to the closest common ancestor when siblings need it.
- Derived state: compute during render, don't store it (`const fullName = first + last` — no `useState/useEffect` for it).

Hooks — what & why each one

Rules: only at top level (no ifs/loops), only in components/custom hooks — because React tracks hooks **by call order**.

Hook	Use for	Key gotcha
<code>useState</code>	Local state	Snapshot semantics; updater form for sequential updates
<code>useEffect</code>	Sync with external systems (fetch, subscriptions, timers, DOM APIs)	Runs AFTER paint. Cleanup runs before next effect & on unmount. Deps array: <code>[]</code> = mount only, <code>none</code> = every render. Stale closures if deps are wrong.
<code>useLayoutEffect</code>	Same, but BEFORE paint	For measuring DOM / preventing visual flicker. Blocks paint — use sparingly.
<code>useRef</code>	Mutable value that survives renders without causing re-render ; DOM access	<code>ref.current = x</code> never re-renders. Uses: DOM nodes, timer ids, previous values, "instance variables"
<code>useMemo</code>	Cache an expensive computed value	Recomputes only when deps change. Don't wrap

Hook	Use for	Key gotcha
		everything — it has its own cost.
<code>useCallback</code>	Cache a function reference	<code>useCallback(fn, deps) ≡ useMemo(() => fn, deps)</code> . Matters when passing callbacks to <code>React.memo</code> children or as effect deps.
<code>useContext</code>	Read nearest Provider's value	ALL consumers re-render when the value changes → split contexts or memoize the value object
<code>useReducer</code>	Complex state transitions / related fields	<code>dispatch({type})</code> → pure reducer returns new state. Good when next state depends on previous + many actions.
<code>useTransition</code>	Mark updates as non-urgent	<code>startTransition(() => setFilter(x))</code> — keeps typing responsive while a heavy list re-renders
<code>useDeferredValue</code>	Deferred copy of a value	Like debouncing a render, not the input
<code>useId</code>	Stable unique ids (SSR-safe)	For label/input htmlFor
Custom hooks	Extract reusable stateful logic	Just a function starting with <code>use</code> that calls other hooks (<code>useDebounce</code> , <code>useFetch</code> , <code>useLocalStorage</code>)

Performance & structure

- `React.memo(Component)`: skip re-render if props are shallow-equal. Breaks if you pass fresh object/function literals → pair with `useMemo/useCallback`.
- Re-render triggers: own state change, parent re-render, context change. (Props changing = parent re-rendered.)

- `React.lazy(() => import('./Page')) + <Suspense fallback={...}>` = code splitting
- Lists: virtualization (react-window) for thousands of rows
- Error boundaries: class component with `componentDidCatch/getDerivedStateFromError` — catch render errors in the subtree (not event handlers/async)
- Portals: `createPortal(children, document.body)` — render outside parent DOM (modals, toasts) while keeping React tree/context
- Controlled (value + onChange from React state) vs uncontrolled (DOM holds value, read via ref) inputs
- StrictMode: dev-only double-invoke of renders/effects to expose impure code and missing cleanups
- Synthetic events: React's cross-browser wrapper, delegated at the root

Rapid-fire React Qs

- Why keys? → identity across renders for reconciliation
 - `useEffect` vs `useLayoutEffect`? → after paint vs before paint
 - `useMemo` vs `useCallback`? → value vs function
 - Why is my state "one step behind"? → snapshot per render / stale closure
 - Context vs Redux/Zustand? → Context is a transport for rarely-changing values (theme, user); not optimized for frequent updates (all consumers re-render)
 - What causes a remount vs update? → element type or key changed vs same type/key
-

4. Next.js

Why Next.js exists

React alone = client-rendered SPA (blank HTML, poor SEO, slow first paint). Next.js adds server rendering, routing, code-splitting, and build-time optimization out of the box.

Rendering strategies (know all four + when)

Strategy	When HTML is made	Use for
CSR	In the browser	Dashboards, authenticated apps

Strategy	When HTML is made	Use for
SSR (dynamic)	On the server, per request	Personalized/frequently changing pages
SSG	At build time	Marketing pages, blogs, docs
ISR	Build time + regenerate after <code>revalidate</code> seconds	Product pages — static speed, periodic freshness

App Router (current model)

- File-system routing under `app/`: `page.tsx` (route UI), `layout.tsx` (shared, persists across navigation, doesn't re-render), `loading.tsx` (auto Suspense fallback), `error.tsx` (auto error boundary), `route.ts` (API endpoint)
- Dynamic: `[id]`, catch-all `[...slug]`, groups (`marketing`) don't affect URL
- **Server Components (default)**: run only on the server, can be async and fetch directly, ship **zero JS** to the client, can access secrets/DB. Can't use state/effects/event handlers.
- **Client Components**: `"use client"` at top. Needed for interactivity — hooks, browser APIs, onClick.
- Pattern: server component fetches data → passes props to small client "islands". Push `"use client"` as deep (leaf-ward) as possible.
- Server components can render client components; a client component can't import a server component (but can receive one as `children`).

Data fetching & caching (App Router)

// in a server component

```
const res = await fetch(url, { cache: 'force-cache' });    // SSG-like
```

```
const res = await fetch(url, { next: { revalidate: 60 } }); // ISR
```

```
const res = await fetch(url, { cache: 'no-store' });      // SSR every request
```

- `revalidatePath` / `revalidateTag` for on-demand invalidation
- Server Actions: `"use server"` functions callable from forms/clients for mutations without hand-writing API routes

Pages Router (legacy — still asked)

- `getServerSideProps` (SSR), `getStaticProps` (+ `revalidate` = ISR), `getStaticPaths` (which dynamic pages to prebuild), `pages/api/*` for API routes

Hydration

Server sends real HTML (fast FCP, SEO) → browser downloads JS → React **attaches event listeners** to existing markup instead of rebuilding it. Hydration mismatch error = server HTML ≠ first client render (e.g., using `Date.now()`, `window`, or random values during render).

Quick hits

- `next/link`: client-side navigation + viewport prefetching
 - `next/image`: lazy loading, resizing, WebP/AVIF, prevents CLS (needs width/height)
 - `next/font`: self-hosted fonts, no layout shift
 - Middleware (`middleware.ts`): runs at the edge before the request — auth redirects, rewrites, headers
 - Streaming: layout can be sent while slow parts stream in via Suspense boundaries
-

5. React Query (TanStack Query)

What problem it solves

Server state ≠ client state. Server data is remote, shared, can go stale. React Query handles fetching, caching, deduping, background refetching, and sync — replacing the `useEffect` + `useState` + loading/error boilerplate.

useQuery — reading data

```
const { data, isLoading, isError, error, isFetching, refetch } = useQuery({  
  
  queryKey: ['todos', userId],      // identity + cache key + dependency  
  
  queryFn: () => fetchTodos(userId),  
  
  staleTime: 60_000,
```

```
});
```

- **queryKey** = the cache identity. Same key anywhere in the app = same cached data (automatic deduping). Include every variable the fetch depends on — key change → refetch.
- **isLoading** = no cached data yet, first load. **isFetching** = any fetch in flight (including background refresh of cached data).
- **enabled: false / !!id** — dependent queries (don't run until id exists)

staleTime vs gcTime (was cacheTime)

- **staleTime** (default 0): how long data is considered FRESH. Fresh → no refetch. Stale → serve cached instantly BUT refetch in background on mount/focus/reconnect.
- **gcTime** (default 5 min): how long UNUSED (no mounted component) data stays in cache before garbage collection.
- Interview one-liner: "staleTime controls when it refetches, gcTime controls when it's deleted."
- Stale-while-revalidate: show cached (possibly stale) data immediately, update quietly → instant UI.

useMutation — writing data

```
const qc = useQueryClient();
```

```
const mutation = useMutation({
```

```
  mutationFn: (newTodo) => axios.post('/todos', newTodo),
```

```
  onSuccess: () => qc.invalidateQueries({ queryKey: ['todos'] } ),
```

```
});
```

```
mutation.mutate({ title: 'x' });
```

Query vs Mutation (the classic question)

	useQuery	useMutation
Purpose	READ (GET)	WRITE (POST/PUT/DELETE)

	useQuery	useMutation
Runs	Automatically on mount / key change	Only when you call <code>.mutate()</code>
Cached?	Yes, by queryKey	No
Refetches on focus/reconnect	Yes (if stale)	Never
Typical follow-up	—	Invalidate related queries so reads refresh

Invalidation & optimistic updates

- `invalidateQueries({queryKey: ['todos']})` → marks matching queries stale → active ones refetch immediately
- Optimistic update flow (know the shape): `onMutate` → cancel outgoing queries, snapshot cache, `setQueryData` to the hoped-for result → `onError` → roll back from snapshot → `onSettled` → invalidate
- `setQueryData` = write cache directly (e.g., from a mutation response) without a refetch

Also worth one line each

- `useInfiniteQuery`: pagination/infinite scroll — `getNextPageParam`, `fetchNextPage`, data comes as `pages[]`
- Retries: failed queries retry 3x with exponential backoff by default
- `refetchOnWindowFocus`: `true` by default (interviewers ask why data "randomly" refreshes)
- Prefetching: `queryClient.prefetchQuery` on hover/route intent
- React Query vs Redux: RQ manages **server cache** (async lifecycle built in); Redux manages **client state**. They complement — RQ often removes the need for most of Redux.

6. Zustand & Redux

Zustand

```
const useStore = create((set, get) => ({
```

```
  count: 0,
```

```

increment: () => set((s) => ({ count: s.count + 1 })),

reset: () => set({ count: 0 }),

});

```

// component — subscribe to a SLICE via selector:

```
const count = useStore((s) => s.count);
```

- One `create()` call → a hook. No Provider needed.
- `set` shallow-merges by default (top level only — nested updates need spreading)
- **Selectors are the perf story**: component re-renders only when its selected slice changes. Selecting the whole store = re-render on every change. Multiple values → `useShallow`.
- `get()` for reading state inside actions; async actions are just async functions that call `set` when done
- Middleware: `persist` (localStorage), `devtools`, `immer`

Redux (Toolkit)

- Principles: single store, state is read-only, changes via **pure reducers**, unidirectional flow: `dispatch(action) → reducer(state, action) → new state → subscribers re-render`
- RTK is the standard now: `configureStore`, `createSlice` (generates action creators + reducer, and lets you write "mutating" code safely via **Immer** — it produces immutable updates under the hood)
- `useSelector(s => s.cart.items)` to read, `useDispatch()` to send actions
- Async: `createAsyncThunk` → handle `pending/fulfilled/rejected` in `extraReducers`; RTK Query for data fetching (Redux's answer to React Query)
- Middleware sits between dispatch and reducer (thunk, logging)

Zustand vs Redux vs Context (comparison they'll ask)

	Context	Zustand	Redux (RTK)
Boilerplate	Low	Minimal	Moderate (RTK helped)
Provider needed	Yes	No	Yes
Re-render control	Poor (all consumers)	Selectors built in	Selectors (<code>useSelector</code>)

	Context	Zustand	Redux (RTK)
DevTools/time travel	No	Via middleware	First-class
Best for	Rarely-changing values (theme, auth user)	Most app state, fast-moving teams	Large apps, strict conventions, complex action flows

- 30-second answer for "why Zustand over Redux?": less boilerplate, no Provider, selector-based subscriptions by default, tiny bundle — while Redux earns its weight in big teams needing enforced structure, middleware ecosystems, and time-travel debugging.

7. Pre-interview quick-fire (test yourself out loud)

CSS: difference between `justify-content` and `align-items`? What does `inset: 0` do? Why animate `transform` instead of `top`? What creates a stacking context?

JS: output order of sync / promise / `setTimeout`? `Promise.all` vs `allSettled` — when each? What's a closure and where does it bite you in React? Debounce vs throttle?

React: walk me through what happens when you call `setState`. What is fiber in 30 seconds? Why not index as key? `useEffect` vs `useLayoutEffect`? How does `React.memo` break and how do you fix it?

Next.js: SSR vs SSG vs ISR — one use case each. Server vs client components — what can each do? What is hydration and what causes a mismatch?

React Query: `staleTime` vs `gcTime`? Query vs mutation? Describe an optimistic update.

State: why Zustand over Context for frequently-updating state? What does Immer do in RTK?

If you can answer all of these out loud in under 30 seconds each, you're ready.

8. JS Fundamentals — Drill Session Reference

Rules, traps, and outputs from the drill. Re-read before any JS round.

Coercion & == (the algorithm)

When types differ, convert in this order:

Step	Rule
1	null == undefined → TRUE. Neither equals anything else.
2	Boolean present → number. false→0, true→1. Repeat.
3	String vs number → string to number. ""→0, "5"→5, "abc"→NaN. Repeat.
4	Object vs primitive → object to string. []→"", [5]→"5", {}→"[object Object]". Repeat.
5	NaN ≠ everything, including itself.

Truthiness vs == — two separate systems. Falsy (only 7): false, 0, -0, "", null, undefined, NaN. Everything else (incl. {} {}) is truthy in if(). But == runs the algorithm above, not truthiness. [] is truthy in if() AND == false. Same value, opposite results.

Expression	Result	Why
[] == false	TRUE	false→0, []→""→0
[1] == 1	TRUE	[1]→"1"→1
null == 0	FALSE	null only == undefined
[] == ""	TRUE	[]→"" same string
"0" == false	TRUE	false→0, "0"→0
[null] == 0	TRUE	[null]→""→0 (null in array = empty str!)
"1" == true	TRUE	true→1, "1"→1 (not about truthiness)
"2" == true	FALSE	true→1, "2"→2, 1≠2

String vs Number Comparison (> < >= <=)

Rule	Detail
Both strings	Lexicographic — char by char. "2">"10"=TRUE ("2">"1"). "B"<"a"=TRUE (uppercase codes < lowercase).
Either is number	Both go numeric. 2>"10" → 2>10=FALSE.
NaN rule	Any comparison with NaN = false. "abc">5 → NaN>5 → false.

Trap: quotes on BOTH sides = forget the number, think dictionary. "10"<"9"=TRUE ("1"<"9").

var — Redeclaration, Hoisting, Loop Closures

Topic	Rule
Redeclaration	var x=1; var x=2 → legal, x=2. let/const → SyntaxError (before code runs, not a runtime error).
Hoisting	var: hoisted as undefined. let/const: hoisted into TDZ → ReferenceError if accessed early.
Dead-branch hoist	var inside if(false){} still hoists the declaration — assignment never runs → undefined, NOT the outer value.
Scope leak	var leaks out of if/for blocks (not functions). i survives after the loop.
Loop exit value	Loop exits when condition FAILS. i<3 → exits at 3. i<5 step+2 → exits at 6. Callbacks see this exit value.
Closure trap	setTimeout runs after loop finishes. All share ONE var i = exit value. let gives each iteration its own binding.

Snippet	Output
for(var i=0;i<3;i++) setTimeout(()=>log(i))	3 3 3
for(let i=0;i<3;i++) setTimeout(()=>log(i))	0 1 2
for(var i=0;i<2;i++){ console.log(i)	2 (leaked out of loop)
function f(){log(z); var z=2} f()	undefined (hoisted local shadows global)
function f(){if(false){var a=2} log(a)} f()	undefined (decl hoists, assign doesn't)

Event Loop — Microtasks vs Macrotasks

Order per cycle: sync → ALL microtasks (drain completely, even newly added) → one macrotask → repeat.

Microtasks: Promise .then/.catch, await continuations, queueMicrotask

Macrotasks: setTimeout, setInterval, I/O

async/await: code before first await = sync. Code after = microtask. Promise executor = sync.

Snippet	Output
log(1); setTimeout(()=>log(2)); P.resolve().then(()=>log(3)); log(4)	1 4 3 2
setTimeout(()=>log('T1')); P.resolve().then(()=>log('P1')).then(()=>log('P2')); log('s')	s P1 P2 T1
new Promise(res=>{log('X');res()}).then(()=>log('Y')); log('Z')	X Z Y
async f(){log('start');await P.resolve();log('after')} setTimeout(()=>log('t')); f(); log('end')	start end after t

Copy Semantics — Shallow vs Deep

Method	Depth	Gotcha
= assignment	None	Same reference — both variables point to same object
spread, Object.assign, slice, Array.from	Shallow	Nested objects still shared — mutating inner leaks to original
structuredClone()	Deep	Can't clone functions. Modern correct answer.
JSON.parse(JSON.stringify())	Deep	Loses functions, undefined; breaks Dates. Name flaws explicitly.

Trap: slice() creates a new outer array but shares inner objects. copy[0].n=99 leaks; copy.push(x) does not. Two ops on same shallow copy can behave differently.

Promise Chaining — Value Flow Rules

Rule	Detail
1 — return	What you return in .then = input to next .then. No return → undefined flows forward.
2 — catch recovers	After .catch runs, chain is healthy. Next .then gets whatever catch returned (undefined if nothing).
3 — throw skips	throw (or reject) skips all .then until nearest .catch. .then invisible in error state; .catch invisible in healthy state.

Snippet	Output
P.resolve(2).then(v=>v*2).then(v=>{log(v)}).then(v=>log(v))	4 undefined (no return = undefined flows)
P.reject('e').then(v=>log('A')).catch(e=>log('B',e)).then(v=>log('C',v))	B e C undefined (A skipped; catch→undefined)
P.resolve(5).then(v=>v+1).then(v=>{log(v);return v*2}).then(v=>log(v))	6 12 (explicit return passes value)
async function f(){return 5} log(f())	Promise {<fulfilled>: 5} (async always returns Promise)

Arrow Function — Brace Rule

Syntax	Returns
v => v + 1	expression (implicit return)
v => { return v + 1 }	expression (explicit return)
v => { console.log(v) }	undefined (braces, no return)
v => { }	undefined

Bites you in .map/.then chains. Unexpected undefined flowing through = check for missing return inside {}.

typeof / instanceof Quick Reference

Expression	Result	Note
typeof null	"object"	Famous JS bug — forever
typeof []	"object"	Arrays are objects
typeof function(){}	"function"	Only callables get this
typeof NaN	"number"	NaN is type number
typeof undefined	"undefined"	
[] instanceof Array	true	
[] instanceof Object	true	Prototype chain: Array → Object

Expression	Result	Note
null instanceof Object	false	null has no prototype chain

Session Quick-Fire (say these out loud before every JS round)

- [] == false? → TRUE. Algorithm not truthiness. false→0, []→""→0.
- "2" > "10"? → TRUE. Both strings → lex. "2">"1".
- 2 > "10"? → FALSE. Number present → numeric. 2>10.
- for(var i<3) setTimeout → 3 3 3. for(let) → 0 1 2.
- var inside if(false) hoists? → YES. Declaration unconditional; assignment is not.
- Promise vs setTimeout order? → Promise (.then = microtask) ALWAYS wins.
- spread = deep copy? → NO. Shallow only — nested objects still shared.
- structuredClone vs JSON trick? → structuredClone is correct. JSON loses functions/undefined/Dates.
- async function returns? → Always a Promise, even if you return 5.
- .catch recovers chain? → YES. Next .then runs with catch's return value.
- typeof null? → "object". typeof NaN? → "number". Both are bugs.
- Arrow {} without return? → undefined flows to next .then.